9：网络编程基础（加点阐述）

任务一：初识网络编程

首先必须要清楚：

两者处于网络通信的不同抽象层级：SocketChannel 是传输层（TCP）的字节流通道，而 HttpURLConnection 是应用层（HTTP）的协议客户端。

联系：HttpURLConnection类的底层本质也是通过SocketChannel的代码实现的，只是对Http协议进行了封装，省去了自己手写请求行、解析状态码的过程。

区别：

	HttpURLConnection	**SocketChannel**
抽象层级	应用层。进行了封装。	传输层。直接面向字节流。
协议支持	由于封装关系，仅支持HTTP/1.1	可用来承载任何协议（但要自己实现）
阻塞模型	完全阻塞	支持非阻塞（好像要配合Selector）
连接复用	有限复用	支持长连接
主要用途	自定义私有协议、需要极低延迟的场景	遗留系统维护

至于现代后端为何不再用HTTPURLConnection:

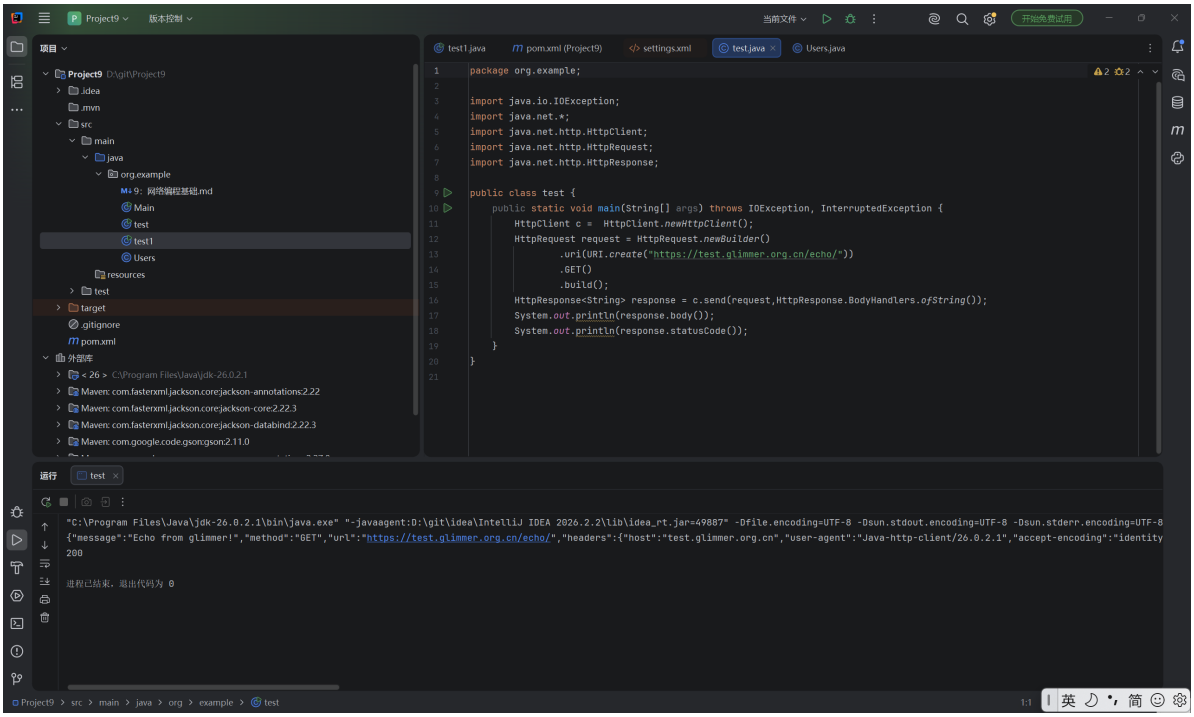
1.API设计陈旧，体验感差：具体来说，就是你连发送一个Header都要用多行代码来实现，且很容易写错。

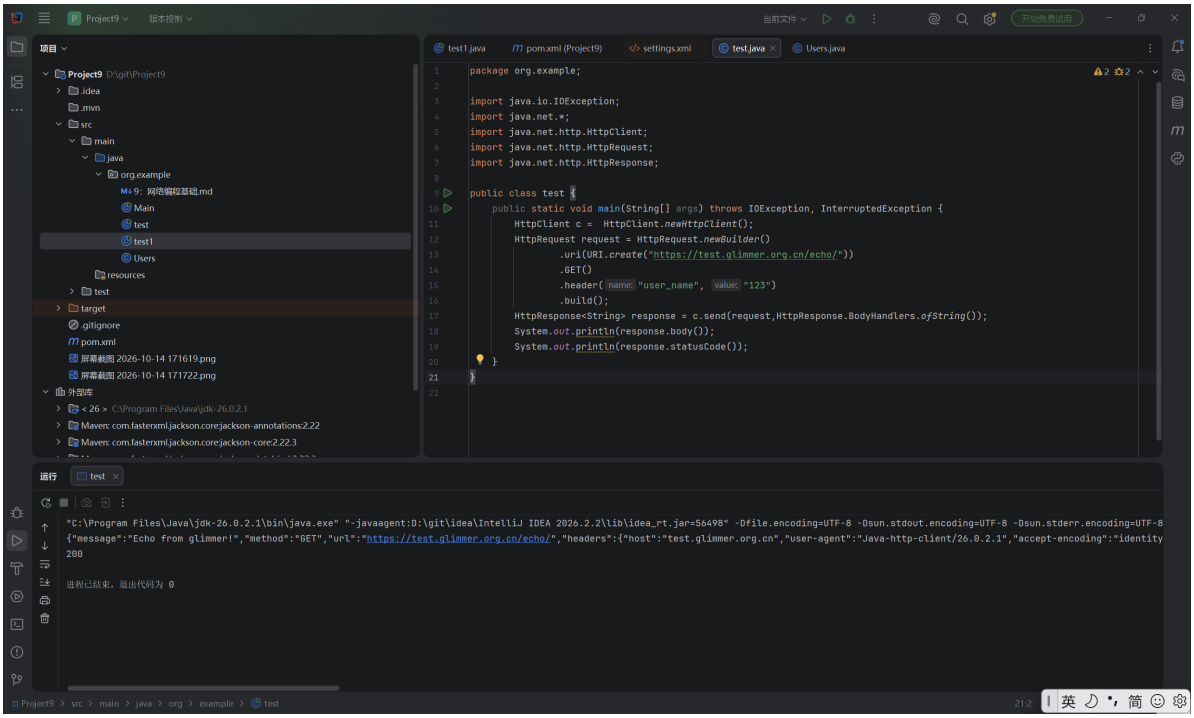
2.是一个阻塞模型：现代后端通常需要用到非阻塞的异步调用来支撑大型数据传输，而它不可能做到

任务二：上手网络编程

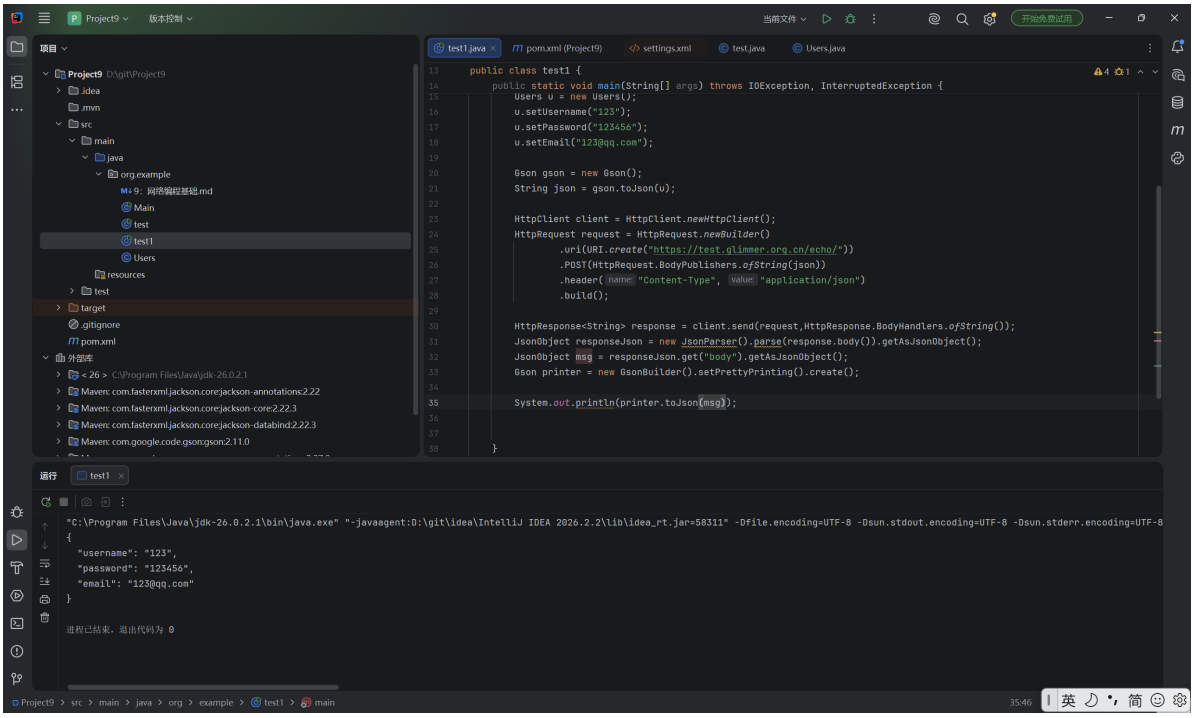
基础实践：API数据交互

不带Json:


Project9
版本控制
当前文件
开始免费试用
项目
test1.java
pom.xml (Project9)
settings.xml
test.java
Users.java
package org.example;
import java.io.IOException;
import java.net.*;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
public class test {
public static void main(String[] args) throws IOException, InterruptedException {
HttpClient c = HttpClient.newHttpClient();
HttpRequest request = HttpRequest.newBuilder()
.uri(URI.create("https://test.glimmer.org.cn/echo/"))
.GET()
.build();
HttpResponse<String> response = c.send(request,HttpResponse.BodyHandlers.ofString());
System.out.println(response.body());
System.out.println(response.statusCode());
}
}
运行
test
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe" "-javaagent:D:\git\idea\IntelliJ IDEA 2026.2.2\lib\idea_rt.jar=49887" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8
{"message":"Echo from glimmer!","method":"GET","url":"https://test.glimmer.org.cn/echo/","headers":{"host":"test.glimmer.org.cn","user-agent":"Java-http-client/26.0.2.1","accept-encoding":"identity
200
进程已结束，退出代码为 0

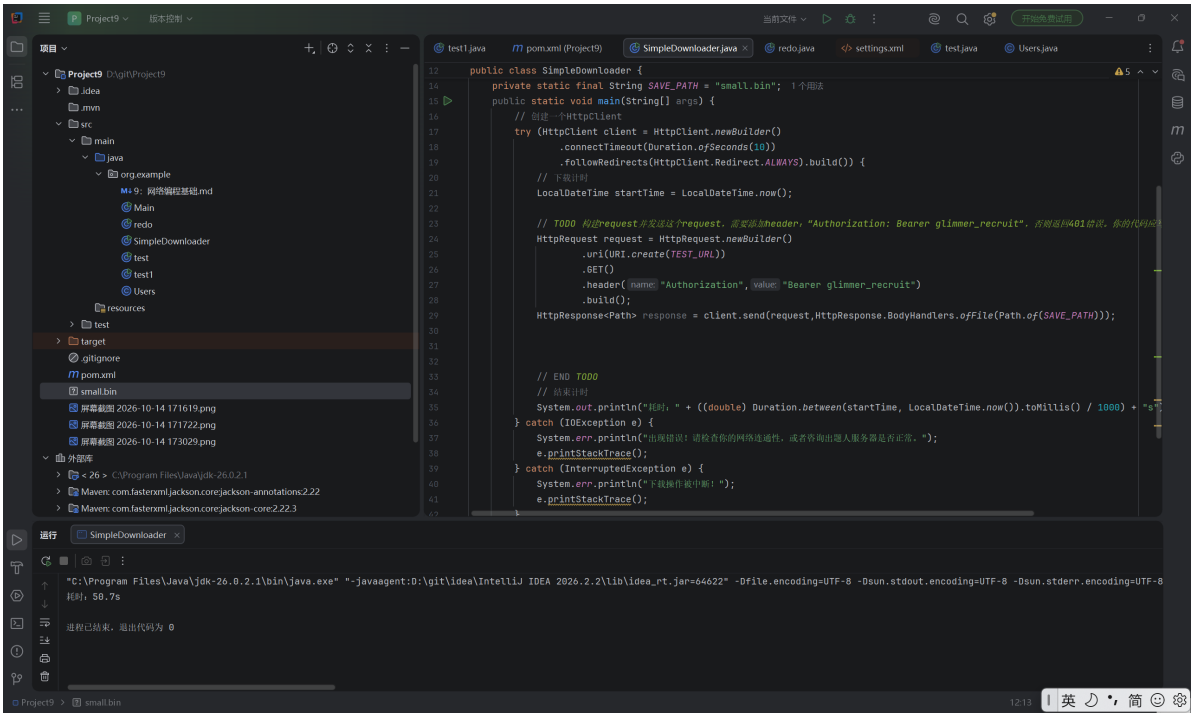

Project9
版本控制
当前文件
开始免费试用
项目
test1.java
pom.xml (Project9)
settings.xml
test.java
Users.java
package org.example;
import java.io.IOException;
import java.net.*;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
public class test {
public static void main(String[] args) throws IOException, InterruptedException {
HttpClient c = HttpClient.newHttpClient();
HttpRequest request = HttpRequest.newBuilder()
.uri(URI.create("https://test.glimmer.org.cn/echo/"))
.GET()
.header(name: "user_name", value: "123")
.build();
HttpResponse<String> response = c.send(request,HttpResponse.BodyHandlers.ofString());
System.out.println(response.body());
System.out.println(response.statusCode());
}
}
运行
test
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe" "-javaagent:D:\git\idea\IntelliJ IDEA 2026.2.2\lib\idea_rt.jar=56498" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8
{"message":"Echo from glimmer!","method":"GET","url":"https://test.glimmer.org.cn/echo/","headers":{"host":"test.glimmer.org.cn","user-agent":"Java-http-client/26.0.2.1","accept-encoding":"identity
200
进程已结束，退出代码为 0

带Json:



(搞了半天还是个回声服务器，还得把body取出来)

挑战实践：实现一个下载器



只需在上一个任务的基础上将响应内的文件提取保存至某个路径即可。

下载文件如下图所示：



任务三：深入网络编程

查询字符串

```
import okhttp3.HttpUrl;

HttpUrl.Builder urlBuilder =
HttpUrl.parse("https://www.example.com").newBuilder();
urlBuilder.addQueryParameter("share_source", "copy_web");
urlBuilder.addQueryParameter("vd_source", "3848787f4de6668e568f67f4434057ac");
String url = urlBuilder.build().toString();
```

使用urlBuilder及其相关方法定制url即可

责任链模式

修改后的代码如下：

```
package org.example;

import com.alibaba.fastjson2.JSON;
import okhttp3.*;

import java.io.IOException;

public class UserManager {
    private static final String host = "https://test.glimmer.org.cn";
    private static final String auth = "Bearer glimmer_recruit";

    public static final OkHttpClient client = new OkHttpClient.Builder()
            .addInterceptor(new Interceptor() {
                @Override
                public Response intercept(Chain chain) throws IOException {
                    Request request = chain.request().newBuilder()
                            .addHeader("Authorization", auth)
                            .build();
```

```
                    return chain.proceed(request);
                }
            })
            .build();

    public record User(long id, String nickname) {}

    public static boolean addUser(User user) {
        Request request = new Request.Builder()
                .url(host + "/users")
                .post(RequestBody.create(JSON.toJSONString(user),
MediaType.parse("application/json")))
                .build();
        return execute(request);
    }

    public static boolean removeUser(long id) {
        Request request = new Request.Builder()
                .url(host + "/users/" + id)
                .delete()
                .build();
        return execute(request);
    }

    public static boolean updateUser(User user) {
        Request request = new Request.Builder()
                .url(host + "/users/" + user.id)
                .put(RequestBody.create(JSON.toJSONString(user),
MediaType.parse("application/json")))
                .build();
        return execute(request);
    }

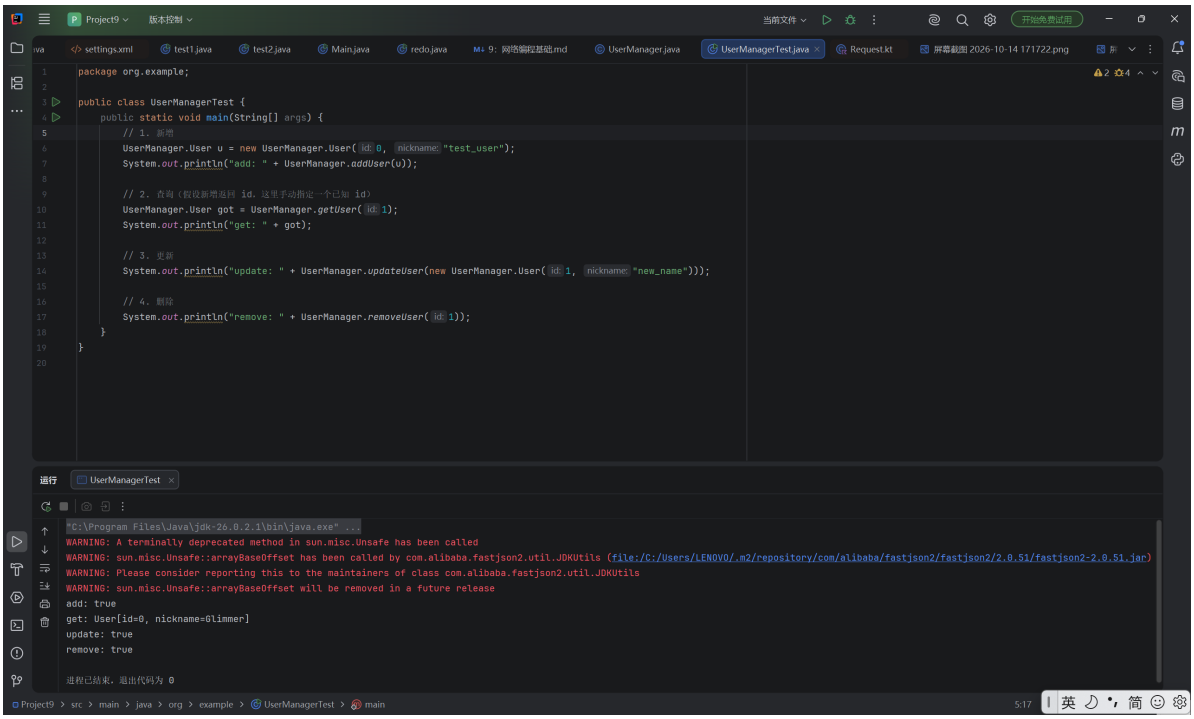
    public static User getUser(long id) {
        Request request = new Request.Builder()
                .url(host + "/users/" + id)
                .get()
                .build();
        try (Response res = client.newCall(request).execute()) {
            if (res.isSuccessful()) {
                ResponseBody body = res.body();
                if (body != null) {
                    return JSON.parseObject(body.string(), User.class);
                }
            }
        } catch (IOException e) {
            System.err.println("请求出现异常，请检查网络可达性");
        }
        return null;
    }

    private static boolean execute(Request request) {
        try (Response res = client.newCall(request).execute()) {
            return res.isSuccessful();
        } catch (IOException e) {
```

```
            System.err.println("请求出现异常，请检查网络可达性");
        }
        return false;
    }
}
```

使用链式编程为客户端的构建添加拦截器，并使用addheader统一添加请求头（但后面的每个方法中用到的客户端对象都必须使用一开始new出来的）

测试截图如下：

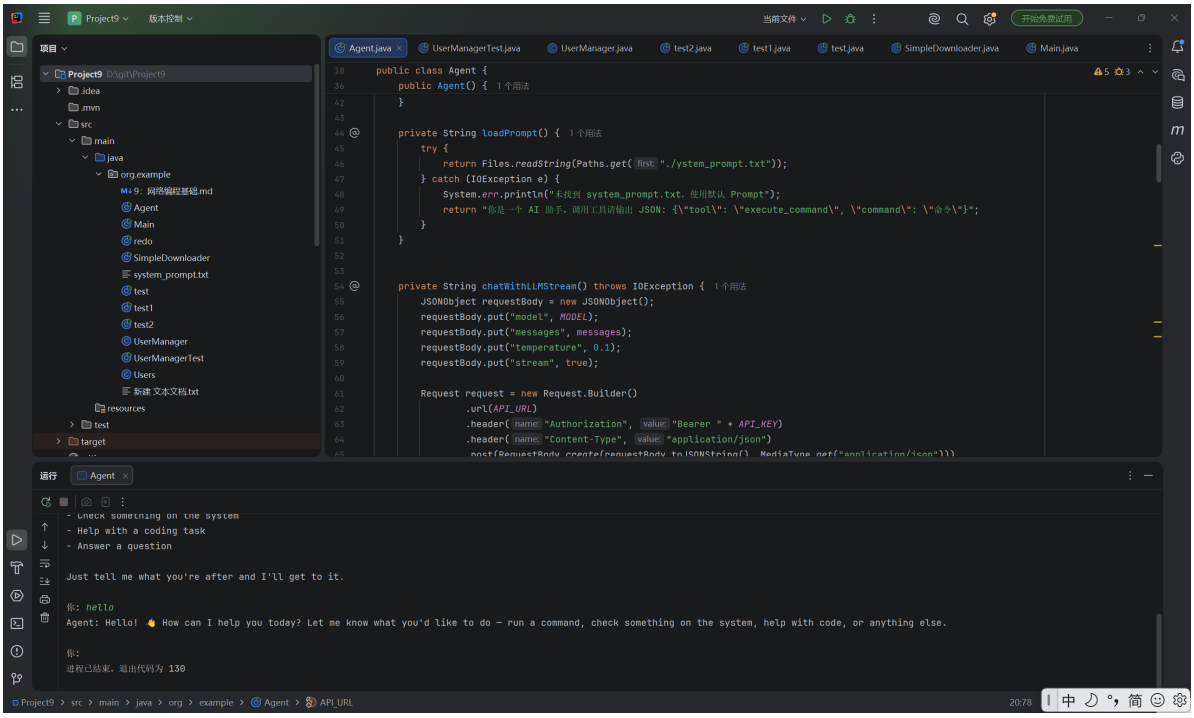


对于思考题，经过我的了解，okhttp中的拦截器interceptor实现的网络IO、重定向、日志、重试、自定义处理等功能拼在一起就像工厂中的流水线一样（我想到了Stream流），可以批量处理传入的请求、响应等数据，即提高了效率，又便于编程，而且具有极高的安全性。

（中间的报错好像是因为数据的安全性问题）

实践：构建你的CLI AI Agent

任务一：



简单补充一下对代码的解释（其实一点都不简单T__T）：

首先声明整个agent模型的基础：模型名称、地址、API密钥

然后搭建客户端框架（设置连接模型与阅读数据的时间），并为对话的上下文做准备（这里用到了JSONArray和JSONObject来储存对话用到的json）,同时传输prompt，建立对话的环境。prompt的导入直接用file.readString搞定，同时捕捉IO报错，并添加一个默认的prompt文本。

最后用while(true)来打造agent核心，使对话持久化。

关于那些加分项，字符跳出的炫酷效果比较简单，在请求中加入"Stream""true"来实现；危险指令的识别与拦截也还好，用正则表达式也能控制；最难的是网络连接部分，查了好久才知道可以用指数退避来实现。

至于一些细节，我为每一次思考与重试设置了最大值，避免ai陷入死循环；针对不同操作系统，我使用了process相关类和一个三元表达式来获取系统类型并判断shell类型；为了赋予ai使用工具的能力，我使用了Stream流来拆解response从而获取并调用该指令（危险指令那里也有）。

其他的异常处理和用户请求读取都相对比较简单。